

EPIC 04- Task 03: Role-Based Authorization

The requirement in your screenshot is:

ADMIN
USER
MANAGER

Restrict APIs based on roles.

We will continue from **Task 02 JWT Authentication** and implement role-based access.

Role permissions

Role	Access
ADMIN	All APIs
MANAGER	Manager and user APIs
USER	User APIs only

Step 1 — Update User.java

Open your existing User.java and add a role field:

```
package com.example.userservice.model;

public class User {

    private Long id;
    private String username;
    private String password;
    private String role;

    public User() {
    }

    public User(Long id, String username, String password, String role) {
        this.id = id;
    }
}
```

```
        this.username = username;
        this.password = password;
        this.role = role;
    }

    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getUsername() {
        return username;
    }

    public void setUsername(String username) {
        this.username = username;
    }

    public String getPassword() {
        return password;
    }

    public void setPassword(String password) {
        this.password = password;
    }

    public String getRole() {
        return role;
    }

    public void setRole(String role) {
        this.role = role;
    }
}
```

Step 2 — Update RegisterRequest.java

Add the role:

```
package com.example.userservice.model;

public class RegisterRequest {

    private String username;
    private String password;
    private String role;

    public String getUsername() {
        return username;
    }

    public void setUsername(String username) {
        this.username = username;
    }

    public String getPassword() {
        return password;
    }

    public void setPassword(String password) {
        this.password = password;
    }

    public String getRole() {
        return role;
    }

    public void setRole(String role) {
        this.role = role;
    }
}
```

Step 3 — Update AuthController.java

When registering a user, store the selected role.

```
package com.example.userservice.controller;

import com.example.userservice.model.LoginRequest;
import com.example.userservice.model.RegisterRequest;
import com.example.userservice.model.User;
import com.example.userservice.security.JwtService;

import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/auth")
public class AuthController {

    private final PasswordEncoder passwordEncoder;
    private final JwtService jwtService;

    private User registeredUser;

    public AuthController(
        PasswordEncoder passwordEncoder,
        JwtService jwtService) {

        this.passwordEncoder = passwordEncoder;
        this.jwtService = jwtService;
    }

    @PostMapping("/register")
    public String register(
        @RequestBody RegisterRequest request) {

        String hashedPassword =
            passwordEncoder.encode(request.getPassword());
```

```

        registeredUser = new User(
            1L,
            request.getUsername(),
            hashedPassword,
            request.getRole()
        );

        return "User registered successfully";
    }

    @PostMapping("/login")
    public String login(
        @RequestBody LoginRequest request) {

        if (registeredUser == null) {
            return "User not registered";
        }

        if (!registeredUser.getUsername()
            .equals(request.getUsername())) {

            return "Invalid username";
        }

        if (!passwordEncoder.matches(
            request.getPassword(),
            registeredUser.getPassword())) {

            return "Invalid password";
        }

        return jwtService.generateToken(
            registeredUser.getUsername(),
            registeredUser.getRole()
        );
    }
}

```

Step 4 — Update JwtService.java

The JWT must contain the user's role.

Replace your existing `JwtService.java` with:

```
package com.example.userservice.security;

import io.jsonwebtoken.Jwts; import io.jsonwebtoken.security.Keys; import
org.springframework.stereotype.Service;

import javax.crypto.SecretKey; import java.util.Date;

@Service public class JwtService {

    private final String SECRET =
        "mySecretKeyForJwtAuthentication12345678901234567890";

    private final SecretKey key =
        Keys.hmacShaKeyFor(SECRET.getBytes());

    public String generateToken(String username, String role) {

        return Jwts.builder()
            .subject(username)
            .claim("role", role)
            .issuedAt(new Date())
            .expiration(
                new Date(
                    System.currentTimeMillis()
                    + 1000 * 60 * 30
                )
            )
            .signWith(key)
            .compact();
    }

    public String extractUsername(String token) {
```

```

return Jwts.parser()
    .verifyWith(key)
    .build()
    .parseSignedClaims(token)
    .getPayload()
    .getSubject();
}

```

```

public String extractRole(String token) {

```

```

    return Jwts.parser()
        .verifyWith(key)
        .build()
        .parseSignedClaims(token)
        .getPayload()
        .get("role", String.class);
}

```

```

public boolean validateToken(String token) {

```

```

    try {
        Jwts.parser()
            .verifyWith(key)
            .build()
            .parseSignedClaims(token);

        return true;

    } catch (Exception e) {
        return false;
    }
}

```

Now the JWT contains:

username
role
expiration

For example:

```
{  
  "sub": "raju",  
  "role": "ADMIN"  
}
```

Step 5 — Create JWT Authentication Filter

Create this package:

com.example.userservice.security

Create:

JwtAuthenticationFilter.java

```
package com.example.userservice.security;  
  
import jakarta.servlet.FilterChain;  
import jakarta.servlet.ServletException;  
import jakarta.servlet.http.HttpServletRequest;  
import jakarta.servlet.http.HttpServletResponse;  
  
import  
org.springframework.security.authentication.UsernamePasswordAuthentica  
tionToken;  
import  
org.springframework.security.core.authority.SimpleGrantedAuthority;  
import  
org.springframework.security.core.context.SecurityContextHolder;  
import org.springframework.web.filter.OncePerRequestFilter;  
  
import java.io.IOException;
```



```
import java.util.List;

public class JwtAuthenticationFilter
    extends OncePerRequestFilter {

    private final JwtService jwtService;

    public JwtAuthenticationFilter(JwtService jwtService) {
        this.jwtService = jwtService;
    }

    @Override
    protected void doFilterInternal(
        HttpServletRequest request,
        HttpServletResponse response,
        FilterChain filterChain)
        throws ServletException, IOException {

        String authHeader =
            request.getHeader("Authorization");

        if (authHeader != null
            && authHeader.startsWith("Bearer ")) {

            String token =
                authHeader.substring(7);

            if (jwtService.validateToken(token)) {

                String username =
                    jwtService.extractUsername(token);

                String role =
                    jwtService.extractRole(token);

                SimpleGrantedAuthority authority =
                    new SimpleGrantedAuthority(
                        "ROLE_" + role
                    );
            }
        }
    }
}
```

```

        UsernamePasswordAuthenticationToken authentication =
            new UsernamePasswordAuthenticationToken(
                username,
                null,
                List.of(authority)
            );

        SecurityContextHolder
            .getContext()
            .setAuthentication(authentication);
    }
}

filterChain.doFilter(request, response);
}
}

```

Step 6 — Update SecurityConfig.java

Replace your SecurityConfig.java with:

```

package com.example.userservice.config;

import com.example.userservice.security.JwtAuthenticationFilter; import
com.example.userservice.security.JwtService;

import org.springframework.context.annotation.Bean; import
org.springframework.context.annotation.Configuration;

import org.springframework.security.config.annotation.web.builders.HttpSecurity; import
org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder; import
org.springframework.security.crypto.password.PasswordEncoder;

import org.springframework.security.web.SecurityFilterChain; import
org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilte
r;

@Configuration public class SecurityConfig {

```

```

private final JwtService jwtService;

public SecurityConfig(JwtService jwtService) {
    this.jwtService = jwtService;
}

@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}

@Bean
public SecurityFilterChain securityFilterChain(
    HttpSecurity http) throws Exception {

    JwtAuthenticationFilter jwtFilter =
        new JwtAuthenticationFilter(jwtService);

    http
        .csrf(csrf -> csrf.disable())

        .authorizeHttpRequests(auth -> auth

            // Public APIs
            .requestMatchers(
                "/api/auth/register",
                "/api/auth/login"
            ).permitAll()

            // ADMIN only
            .requestMatchers(
                "/api/admin/**"
            ).hasRole("ADMIN")

            // ADMIN + MANAGER
            .requestMatchers(
                "/api/manager/**"
            ).hasAnyRole("ADMIN", "MANAGER")

```

```

        // ADMIN + MANAGER + USER
        .requestMatchers(
            "/api/user/**"
        ).hasAnyRole(
            "ADMIN",
            "MANAGER",
            "USER"
        )

        // Everything else requires login
        .anyRequest().authenticated()
    )

    .addFilterBefore(
        jwtFilter,
        UsernamePasswordAuthenticationFilter.class
    );

    return http.build();
}
}

```

Step 7 — Create Admin API

Create:

AdminController.java

```

package com.example.userservice.controller;

import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/admin")
public class AdminController {

    @GetMapping("/dashboard")

```

```
        public String adminDashboard() {  
            return "Welcome Admin";  
        }  
    }  
}
```

Only ADMIN can access:

GET /api/admin/dashboard

Step 8 — Create Manager API

Create:

```
ManagerController.java  
package com.example.userservice.controller;  
  
import org.springframework.web.bind.annotation.*;  
  
@RestController  
@RequestMapping("/api/manager")  
public class ManagerController {  
  
    @GetMapping("/dashboard")  
    public String managerDashboard() {  
        return "Welcome Manager";  
    }  
}
```

ADMIN and MANAGER can access:

GET /api/manager/dashboard

Step 9 — Create User API

Create:

```
UserController.java
package com.example.userservice.controller;

import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/user")
public class UserController {

    @GetMapping("/profile")
    public String profile() {
        return "User profile";
    }
}
```

ADMIN, MANAGER, and USER can access:

GET /api/user/profile

Step 10 — Test in Postman

Register ADMIN

POST <http://localhost:8080/api/auth/register>

Body:

```
{
  "username": "admin",
  "password": "12345",
  "role": "ADMIN"
}
```

Then login:

POST <http://localhost:8080/api/auth/login>

```
{
```

```
"username": "admin",  
"password": "12345"  
}
```

Copy the JWT returned by the login API.

Test Admin API

In Postman:

GET <http://localhost:8080/api/admin/dashboard>

Headers:

Authorization: Bearer YOUR_JWT_TOKEN

Response:

Welcome Admin

Test USER role

Register/login with:

```
{  
  "username": "raju",  
  "password": "12345",  
  "role": "USER"  
}
```

The USER can access:

GET /api/user/profile

but cannot access:

GET /api/admin/dashboard

The server should return:

403 Forbidden

Final structure

Your Eclipse project should now look like:

src/main/java

```
└─ com.example.userservice
    |
    │── config
    │   └─ SecurityConfig.java
    |
    │── controller
    │   ├── AuthController.java
    │   ├── AdminController.java
    │   ├── ManagerController.java
    │   └─ UserController.java
    |
    │── model
    │   ├── User.java
    │   ├── LoginRequest.java
    │   └─ RegisterRequest.java
    |
```


└─ security

└─ JwtService.java

└─ JwtAuthenticationFilter.java

Task 03 is complete when:

ADMIN → Admin + Manager + User APIs

MANAGER → Manager + User APIs

USER → User APIs only

and unauthorized role access returns **403 Forbidden**.